NYC Subway Delay Prediction System

Submitted by :

Nydia Takhellambam (25310116)

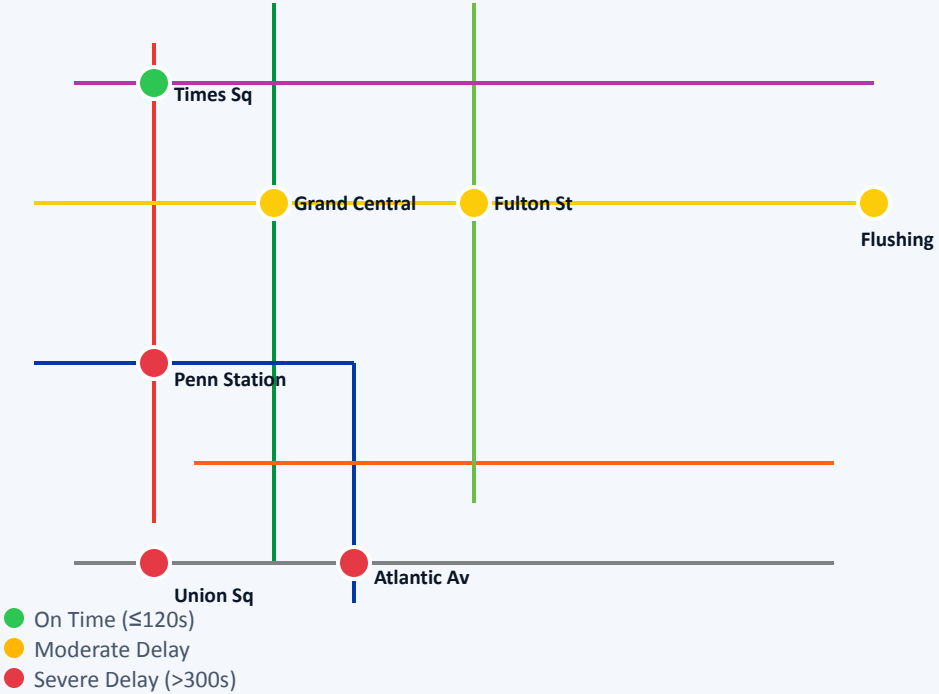
Abhirohan Kashyap (25210004)

Kuntal Maity (25210079)

THE PROBLEM

Why NYC Subway Delays Matter

NYC SUBWAY NETWORK



NYC Transit by the Numbers

5.5M+
Daily Riders

472
Stations

245mi
Track Length

24/7
Service Hours

One of the few transit systems in the world that never closes

9th busiest rapid transit system globally

Cascading delays are the primary operational challenge

PROJECT GOALS

Build a machine learning system that predicts NYC subway delays in seconds using temporal, sequential, and environmental features

01
Data Collection & Preprocessing

- Collect real-time GTFS feeds for over ~4 days
- Collects train positions for every 30s
- Compute delay values
- 37M+ row dataset

02
Feature Engineering

- Time-based features (hour, peak)
- hour (0–23) — captures rush-hour peaks
- prev_delay — delay at immediately preceding stop (same trip)
- rolling_delay — 3-stop rolling average of delay
- cumulative_delay — total accumulated delay on this trip

03
ML Model & Dashboard

- Train XGBoost
- Deploy interactive dashboard
- Enable real-time prediction

Data Sources & Pipeline

The Processing Chain

Real-Time GTFS
500K rows

→

Static GTFS
562K rows

→

Merge & Align
639M → 37M

→

Feature Eng.
20 Features

→

ML-Ready
300K Samples

→

Train ML model

Data Sources

Real-Time GTFS
MTA live feed — actual arrival times, trip IDs, stop IDs

GTFS Schedule
stops.txt, stop_times.txt, trips.txt, routes.txt

Dataset Statistics

37.6M
Rows after merge/filter

300K
Sampled for training

20
Engineered features

58
Stop IDs matched

Preprocessing Pipeline & Data Challenges

Challenges Encountered:

Data Integration Challenge

Real-time GTFS & static GTFS use inconsistent stop_id formats — required string normalization

Merge Explosion Problem

Naive join on stop_id → 639M rows due to many-to-many relationships. Required trip+time key

No Direct Delay Variable

Delay had to be computed: Delay = Actual Arrival − Scheduled Arrival

Extreme Outliers

Delays outside [-600, 3600] seconds filtered; records with missing timestamps removed

5-Step Preprocessing Pipeline:

1. **Data Cleaning**
   Remove duplicates · Convert timestamps · Normalize stop_id strings
2. **Data Integration**
   Merge real-time + GTFS on trip_id + stop_id · 639M → 37.6M rows
3. **Target Variable Creation**
   Delay = Actual − Scheduled · Valid range: −600 to 3600 seconds
4. **Noise Reduction**
   Remove outliers · Filter invalid records · Handle missing values
5. **Feature Engineering**
   Create 20 features: timestamp, stop, prev_delay, rolling_delay , sequential

FEATURE ENGINEERING

⏱️ Time-Based Features

- hour — hour of day (0–23)
- day_of_week — Monday=0 … Sunday=6
- is_peak — rush hour flag (7–9AM, 5–7PM)
- is_weekend — weekday vs weekend

💡 *Captures rush-hour surge patterns and weekly trends*

🔁 Sequential Features

- prev_delay — delay of previous stop
- rolling_delay — rolling average (window=3)
- cumulative_delay — sum of delays on trip
- stop_sequence — position in route

💡 *Models delay propagation across stations (most powerful!)*

🔢 Behavioral Features

- delay (target) — actual − scheduled
- Sorted chronologically per trip
- Missing values filled for continuity
- Non-numeric columns removed for ML

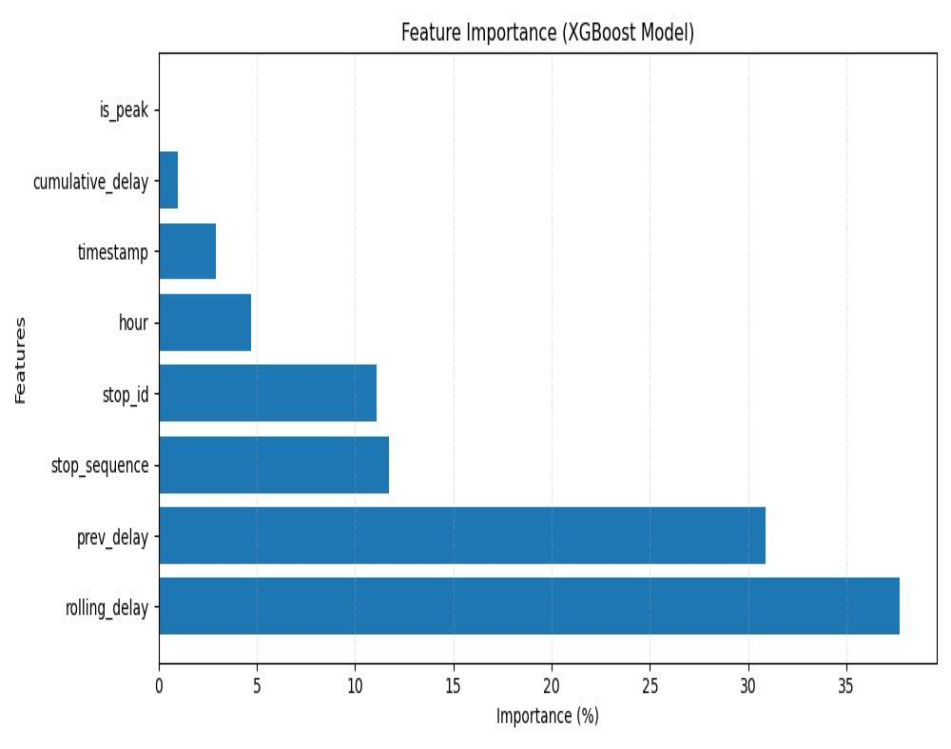
💡 *Ensures model receives clean numerical input only*

Key Insight: Sequential delay features were the strongest predictors — delays propagate from station to station

MODEL SELECTION & TRAINING

Why XGBoost? How We Trained It.

Other Models (Such as LR, Decision Tree etc.) - Linear Relationship, Prone to Overfitting.



Training Strategy

Chronological Train/Test Split (prevents data leakage)

TRAIN — 240,000 rows (80%) | TEST — 60,000 rows (20%)

◂ Earlier in time ———————————— Later in time ▸

Baseline vs XGBoost

Model	MAE	RMSE	R^2
Mean Baseline	1048s	1210s	-0.00
XGBoost	224s	467s	0.85

Model Performance & Interpretation

0.885
R^2 Score
Explains 85% of delay variance

224s
MAE
Mean ~3.7 min prediction error

467s
RMSE
Penalized error for large misses

300K
Training Samples
Chronological 80/20 split

Interpretation

R^2 = 0.85 is Strong

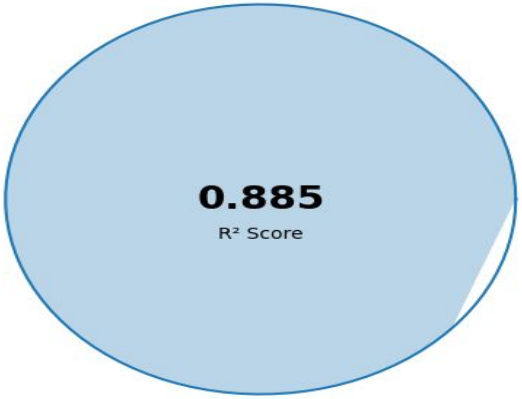
The model explains 85% of delay variability. For a noisy urban transit system, this is excellent performance.

prev_delay Dominates (38%)

The strongest predictor — delays at one station propagate to the next. Temporal context is critical.

MAE ≈ 3.7 minutes

On average, predictions are off by ~3.7 minutes — useful for commuter planning apps and real-time alerts.



Assumptions, Limitations & Future Work

ASSUMPTIONS MADE

Route Prediction: Uses sequential approximation, not full GTFS routing.

Sampling: 300K representative sample of the full 37M row distribution.

Scenario Mirroring: Chronological split mirrors real-world deployment scenarios.

CURRENT LIMITATIONS

API Integration: Uses static GTFS snapshot; no real-time MTA API integration.

GTFS Sequencing: stop_times.txt routing not fully integrated yet.

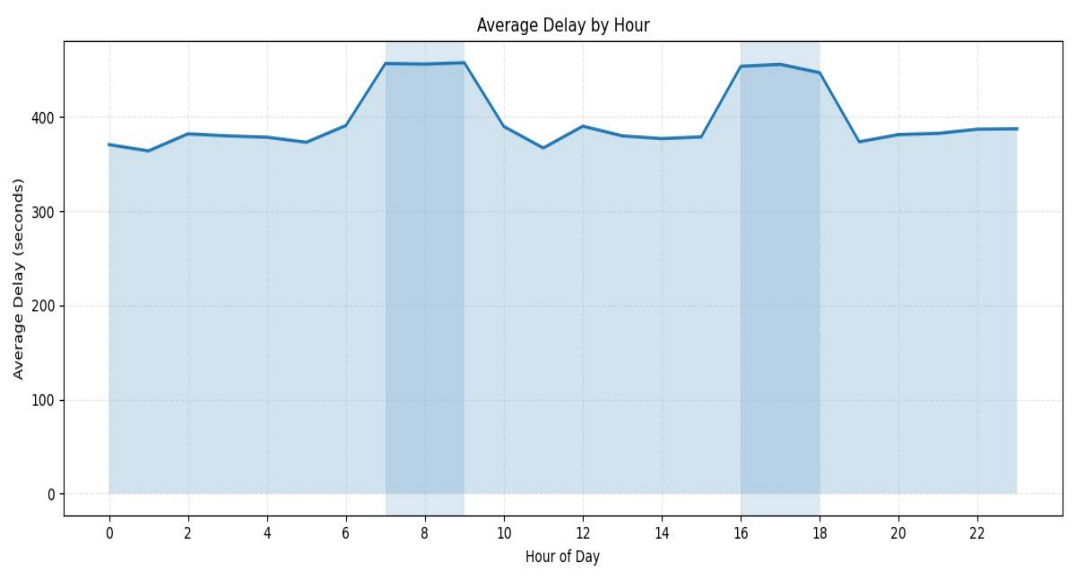
Live Feed: Dashboard runs on sample_data.csv (1K rows), not live feeds.

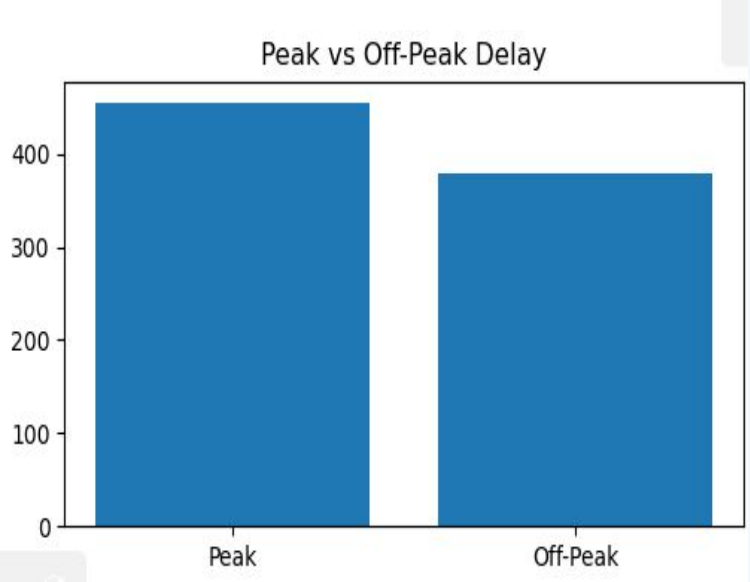
FUTURE WORK

- Real-Time MTA Feeds
- LSTM / Time Series Models
- Mobile-Friendly UI

DISCUSSION

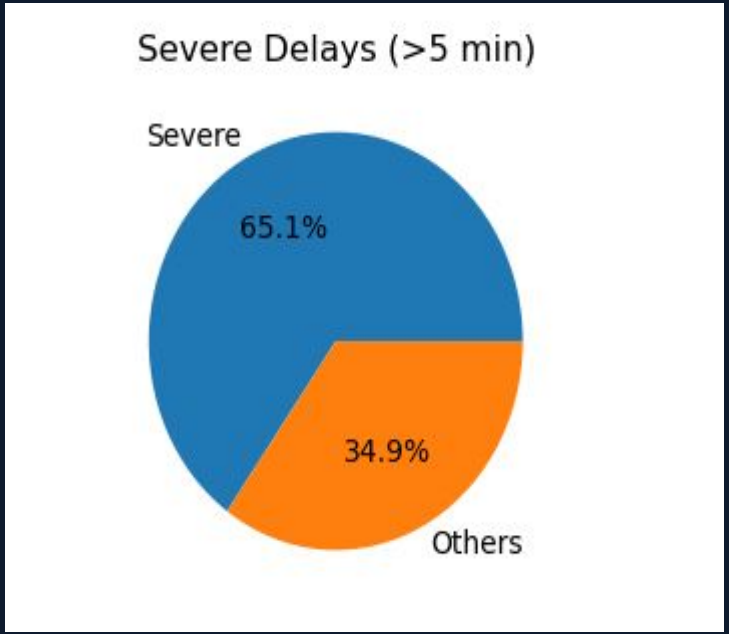
Average Delay by Hour





- **Congestion Patterns:** Delays show clear spikes during **morning (7–9 AM) and evening (4–6 PM) peak hours**, confirming congestion-driven patterns.
- **Operational Stress:** Peak-hour delays are **significantly higher than off-peak**, indicating increased passenger load and operational stress.

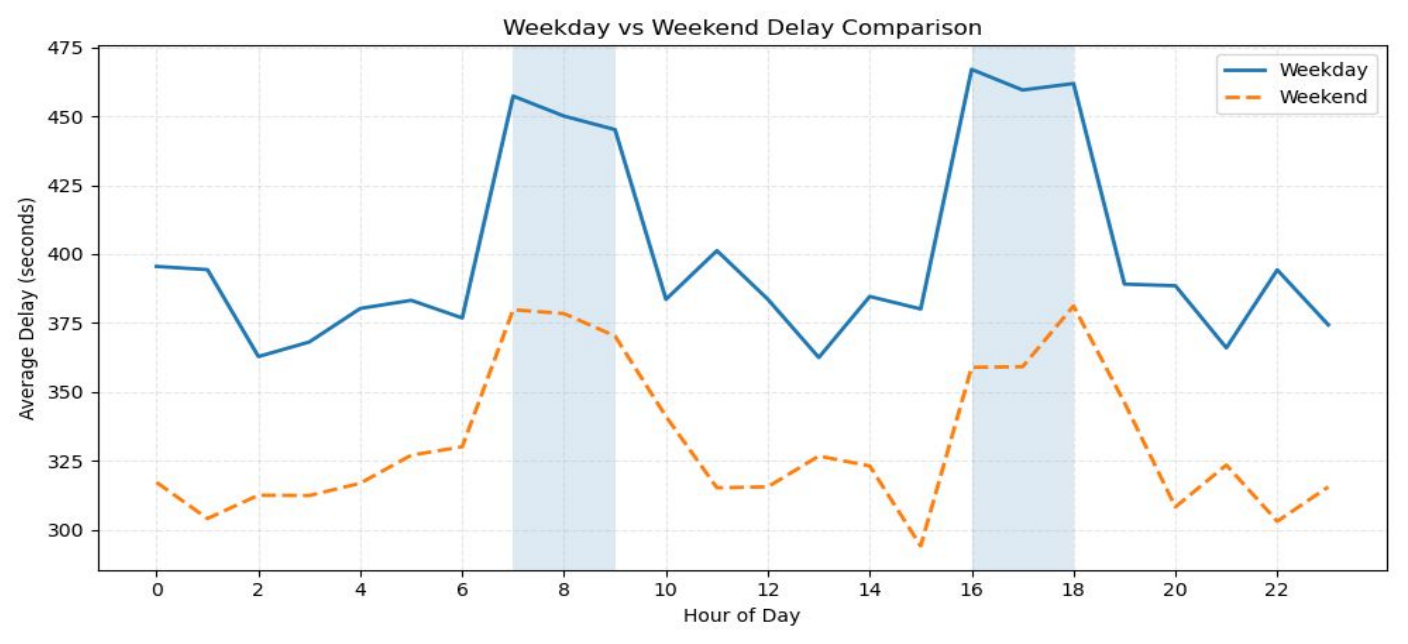
Comparison Average Delay by Hour



- Only a **small fraction of trips are on-time (~7.6%)**, indicating overall system reliability challenges.
- A significant portion of delays are **severe (>5 min ~65%)**, highlighting frequent major disruptions.
- These metrics show that delays are not just frequent but also **intense, reinforcing the need for predictive modeling and optimization**.

DISCUSSION

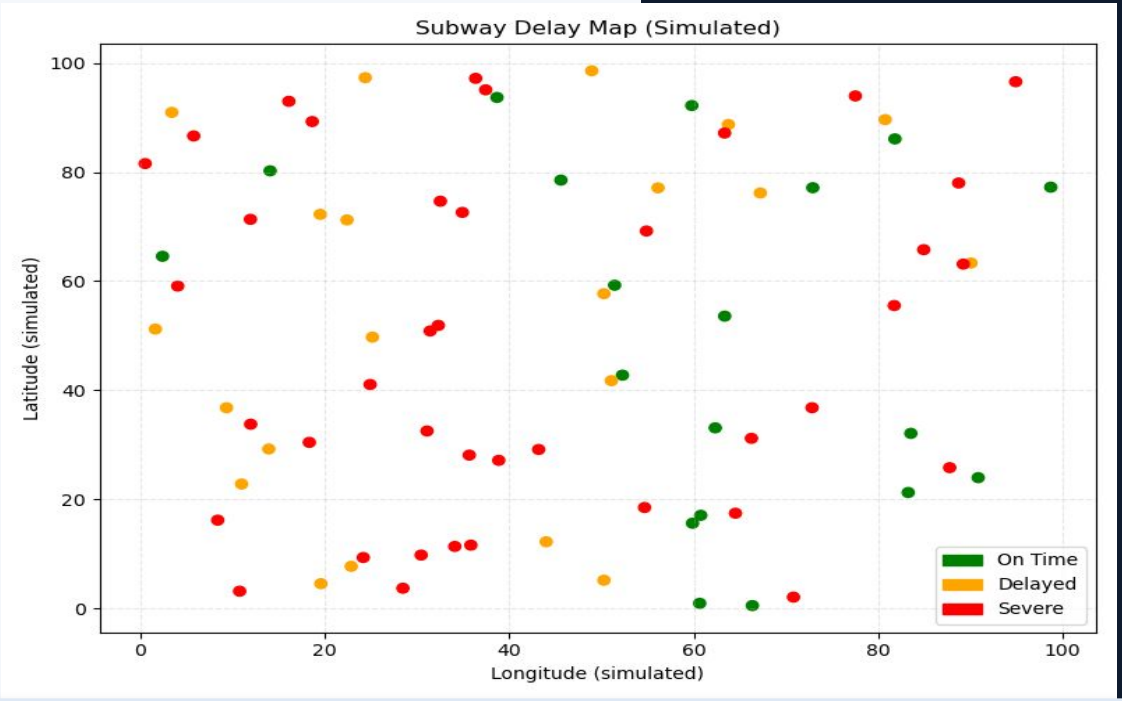
Weekday v/s Weekend Delay Comparison



- Weekday delays are consistently **higher than weekends**, indicating increased demand and congestion during working days.
- Both show peaks during **rush hours**, but the intensity is significantly greater on weekdays.
- Highlights that **day-type is an important factor**, with reduced operational stress and smoother flow on weekends.

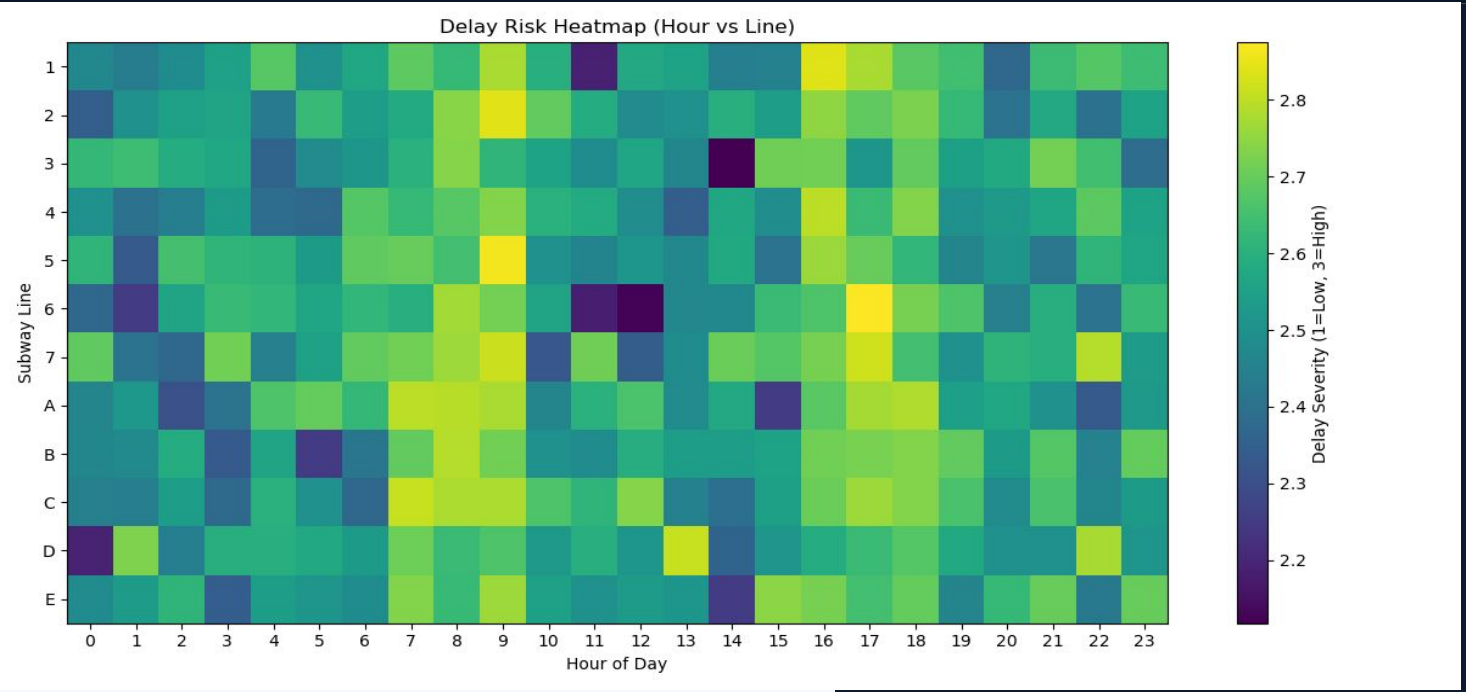
DISCUSSION

Subway Delay Map



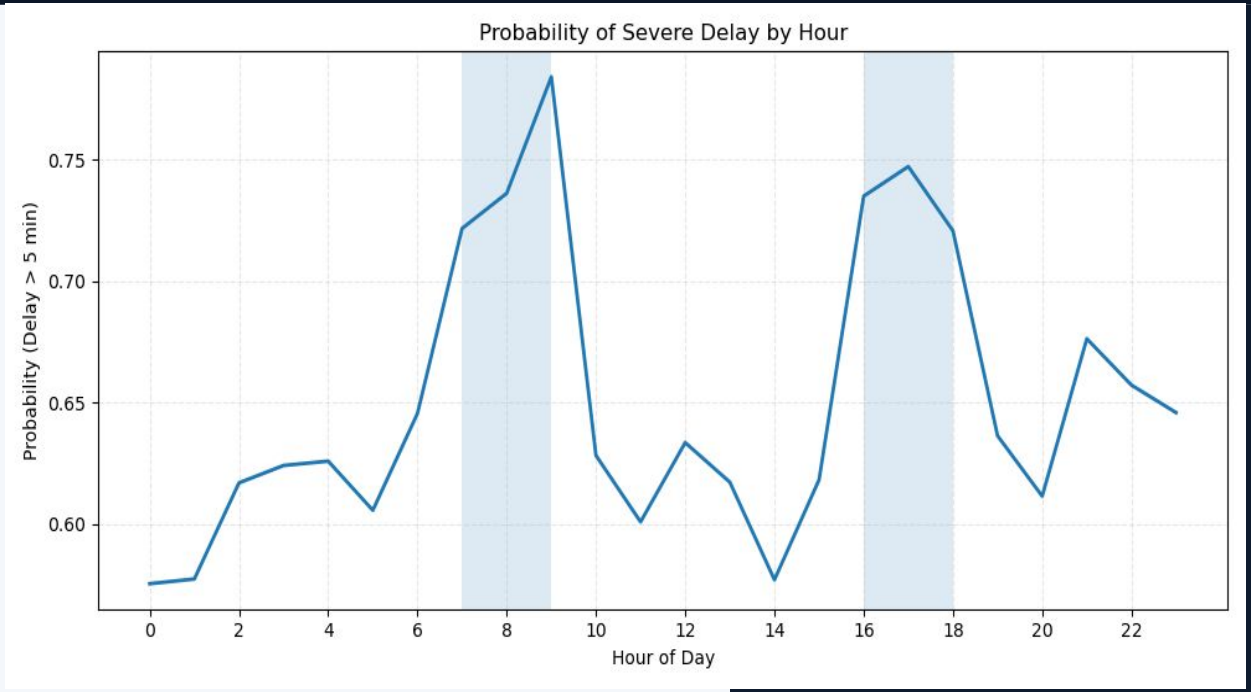
- Delay severity is **spatially distributed across stations**, with multiple hotspots showing severe delays.
- No uniform pattern — delays are **clustered in specific regions**, indicating localized congestion issues.
- Helps identify **high-risk zones**, enabling targeted interventions rather than system-wide changes.

Delay Risk Heat Map (Hour vs Line)



- Delay severity varies across both **time and subway lines**, showing that not all lines are equally affected.
- Peak hours (morning & evening) exhibit **higher risk levels across multiple lines**, confirming system-wide congestion periods.

Probability of Severe Delay by Hour



- Probability of severe delays **peaks during rush hours (7–9 AM and 4–6 PM)**, indicating highest risk periods.
- Even outside peak hours, the probability remains **consistently high**, showing persistent system inefficiencies.
- Provides a **risk-based perspective**, helping commuters and planners anticipate and avoid high-delay periods.

INTERACTIVE DASHBOARD

NYC Subway Delay Predictor

Dashboard Features

Thank you.